```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import keras
import tensorflow as tf
from keras.models import Sequential
from keras.layers import LSTM, Dense
from keras.optimizers import SGD
from tensorflow.keras.layers import Dropout
from matplotlib import pyplot as plt
import plotly.express as px
from plotly.subplots import make_subplots
import plotly.graph_objects as go
import plotly.figure_factory as ff
```

```
# Load the dataset

df = pd.read_csv('discharge.csv')
```

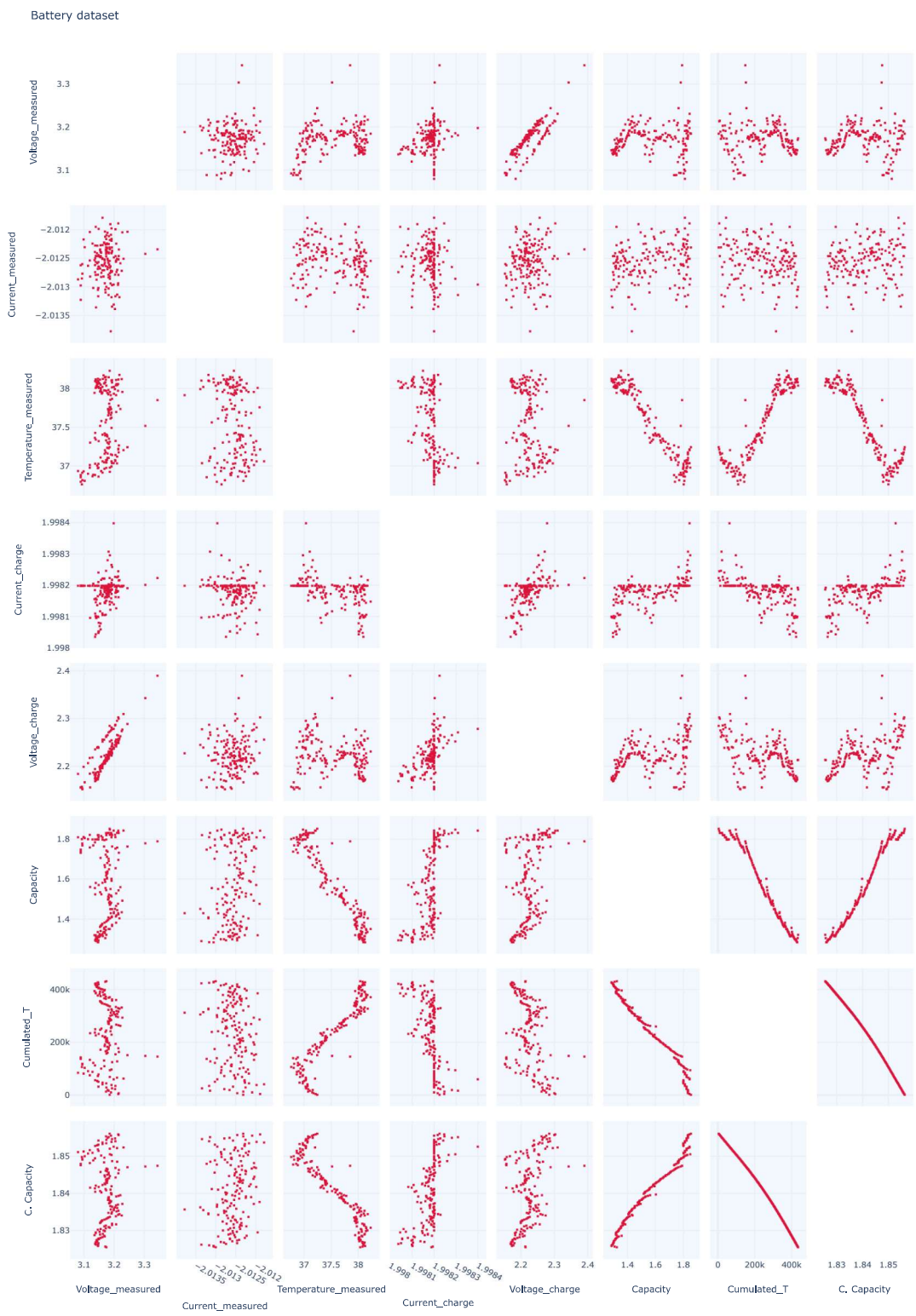
```
df = df[df['Battery'] == 'B0005']
df = df[df['Temperature_measured'] > 36]  # choose battery B0005

# ✅ FIXED VERSION
dfb = df.groupby('id_cycle').mean(numeric_only=True)

dfb['Cumulated_T'] = dfb['Time'].cumsum()
cycles = dfb.index.values
```

```
K = 0.0001
L_e = 1 - np.exp(-K * cycles)

X_in_e = dfb['Capacity'].iloc[0] * (1 - L_e)
K = 0.0001
L_1 = 1 - np.exp(-K * dfb.index)

dfb['C. Capacity'] = dfb['Capacity'].iloc[0] * (1 - L_1)
```

```
from sklearn.preprocessing import StandardScaler

# Step 1: build features
X_in = dfb[['C. Capacity', 'Temperature_measured', 'Time']].copy()
X_in['Cycle'] = dfb.index

# Step 2: scale features (VERY IMPORTANT)
scaler = StandardScaler()
X_in = scaler.fit_transform(X_in)

# Step 3: output (residual)
X_out = (dfb['Capacity'] - dfb['C. Capacity']).values
```

```
import plotly.express as px

cols_to_drop = [col for col in ['Time', 'ambient_temperature', 'time'] if col in dfb.columns]

fig = px.scatter_matrix(
    dfb.drop(columns=cols_to_drop)
)

fig.update_traces(
    marker=dict(size=2, color='crimson', symbol='square'),
    diagonal_visible=False
)

fig.update_layout(
    title='Battery dataset',
    width=900,
    height=1200,
    plot_bgcolor='#f2f8fd',
    paper_bgcolor='white',
    template='plotly_white',
```

```
        font=dict(size=7)
    )

    fig.show()
```

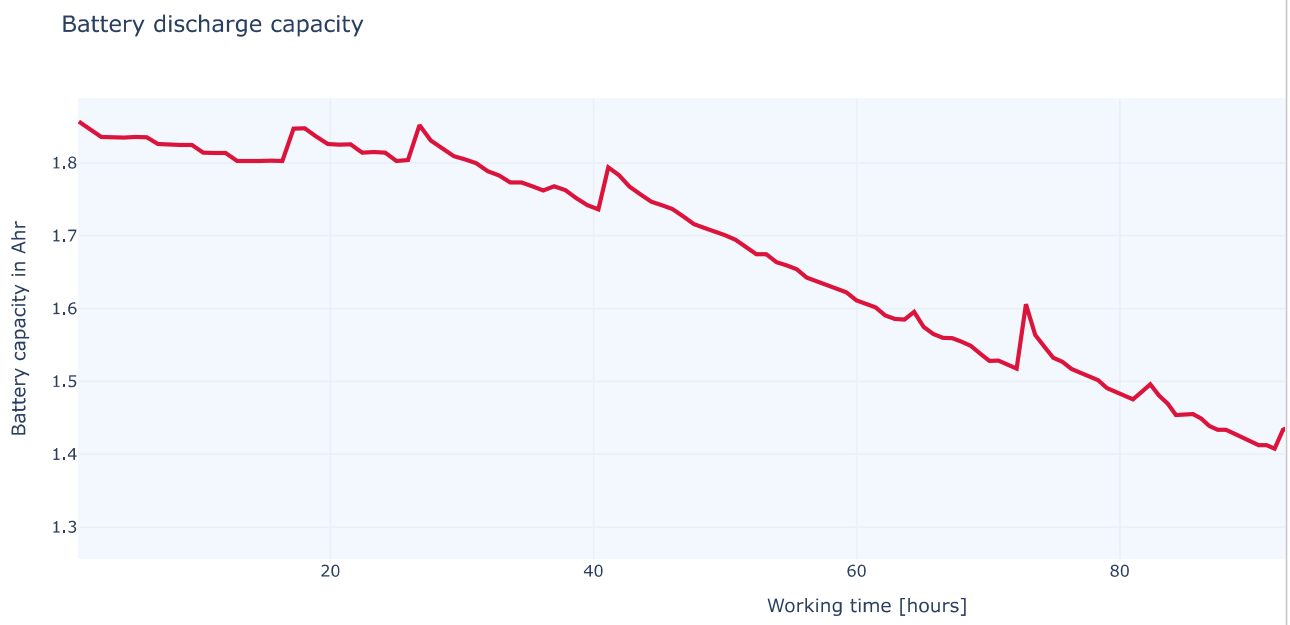


X_in = data['predicted_capacity'].values.reshape(-1, 1) X_out = data['capacity'].values - data['predicted_capacity'].values

X_in_train, X_in_test, X_out_train, X_out_test = train_test_split(X_in, X_out, test_size=0.33)

```
    fig = go.Figure()

    fig.add_trace(go.Scatter(x=dfb['Cumulated_T']/3600,
```

```
                         y=dfb['Capacity'],
                         mode='lines',
                         name='Capacity',
                         marker_size=3,
                         line=dict(color='crimson', width=3)
                        ))
fig.update_layout(
                  title="Battery discharge capacity",
                  xaxis_title="Working time [hours]",
                  yaxis_title=f"Battery capacity in Ahr"
    )
fig.update_layout({'plot_bgcolor': '#f2f8fd',
                   'paper_bgcolor': 'white',},
                   template='plotly_white')
```

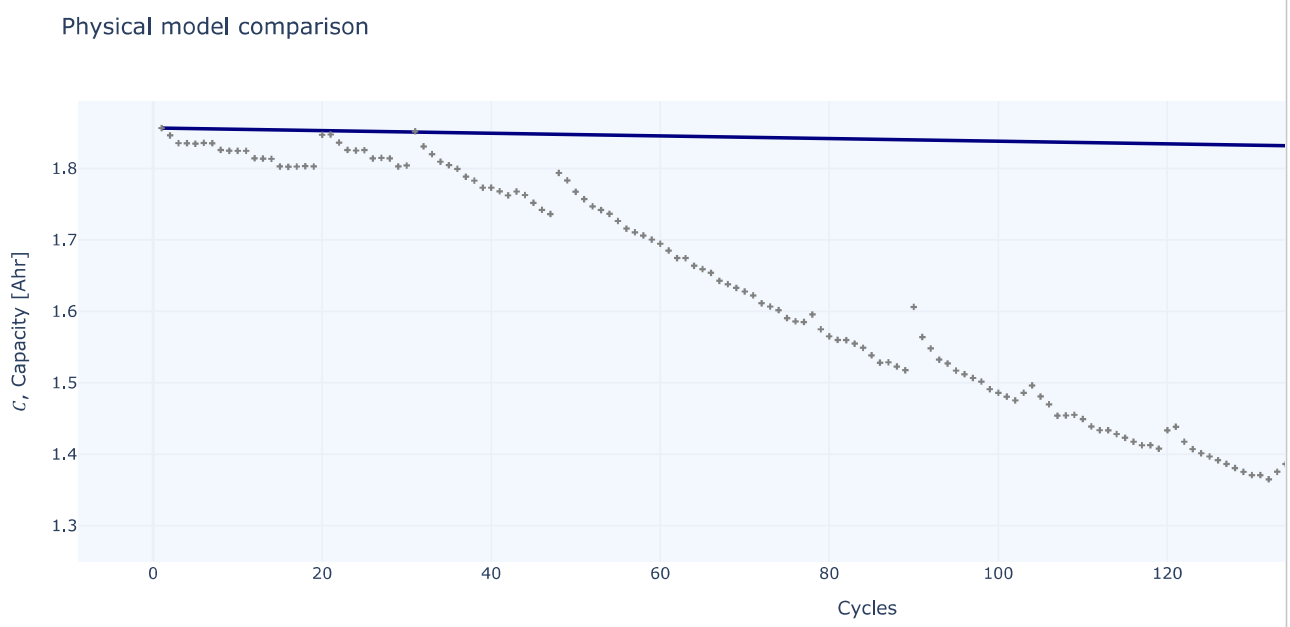



```
K = 0.0001
L_1 = 1 - np.exp(-K * dfb.index)

dfb['C. Capacity'] = dfb['Capacity'].iloc[0] * (1 - L_1)
```

```
# 1. Features
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

features = dfb[['C. Capacity', 'Temperature_measured', 'Time']].copy()
features['Cycle'] = dfb.index

scaler = StandardScaler()
X_in = scaler.fit_transform(features)

X_out = (dfb['Capacity'] - dfb['C. Capacity']).values

X_in_train, X_in_test, X_out_train, X_out_test = train_test_split(
    X_in, X_out, test_size=0.33, random_state=42
)
```

```
fig = go.Figure()

fig.add_trace(go.Scatter(x=dfb.index,
                         y=dfb['C. Capacity'],
                         mode='lines',
                         name='Physical model',
                         line=dict(color='navy',
                                   width=2.5,
                                   )))
```

```
fig.add_trace(go.Scatter(x=dfb.index,
                         y=dfb['Capacity'],
                         mode='markers',
                         marker=dict(
                              size=4,
                              color='grey',
                              symbol='cross'
                                 ),
                         name='NASA dataset',
                         line_color='navy'))
fig.update_layout(
    title="Physical model comparison ",
    xaxis_title="Cycles",
    yaxis_title="$C$, Capacity [Ahr]")

fig.update_layout(legend=dict(
    yanchor="top",
    y=0.9,
    xanchor="left",
    x=0.8
))

fig.update_layout({'plot_bgcolor': '#f2f8fd',
                   'paper_bgcolor': 'white',},
                    template='plotly_white')
```




```
from sklearn.metrics import mean_absolute_error

mae_physical = mean_absolute_error(dfb['Capacity'], dfb['C. Capacity'])

print("Physical Model MAE:", round(mae_physical, 3))
```

```
Physical Model MAE: 0.268
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

features = dfb[['C. Capacity', 'Temperature_measured', 'Time']].copy()
features['Cycle'] = dfb.index

scaler = StandardScaler()
X_in = scaler.fit_transform(features)

X_out = (dfb['Capacity'] - dfb['C. Capacity']).values

X_in_train, X_in_test, X_out_train, X_out_test = train_test_split(
```

```
    X_in, X_out, test_size=0.33, random_state=42
)
```

```
X_in_train.shape
```

```
(112, 4)
```

```
model = Sequential()

model.add(Dense(128, activation='relu', input_shape=(X_in.shape[1],)))
model.add(Dense(128, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(1))

model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

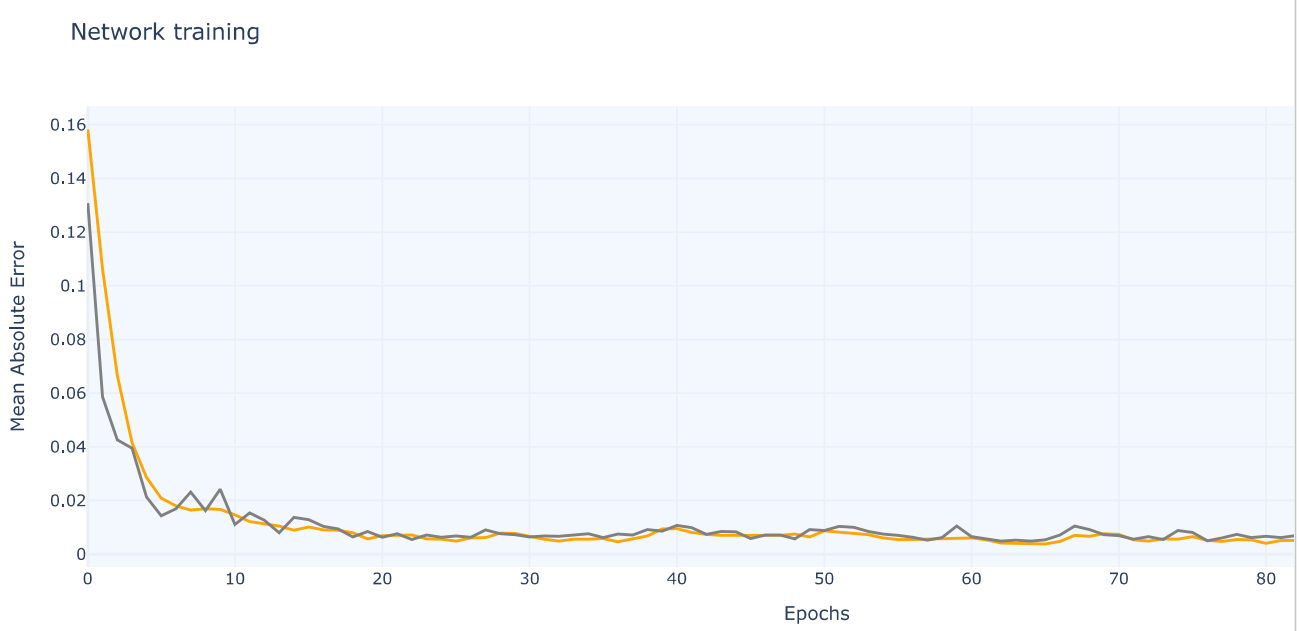
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:

Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` objec
```

```
epochs = 100
loss = "mse"
model.compile(optimizer='adam',
              loss=loss,
              metrics=['mae'], #Mean Absolute Error
             )
history = model.fit(X_in_train, X_out_train,
                    shuffle=True,
                    epochs=epochs,
                    batch_size=20,
                    validation_data=(X_in_test, X_out_test),
                    verbose=1)
```

```
Epoch 1/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 2s 58ms/step - loss: 0.0426 - mae: 0.1583 - val_loss: 0.0241 - val_mae: 0.1309
Epoch 2/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 0.0170 - mae: 0.1062 - val_loss: 0.0062 - val_mae: 0.0586
Epoch 3/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - loss: 0.0069 - mae: 0.0666 - val_loss: 0.0037 - val_mae: 0.0427
Epoch 4/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0031 - mae: 0.0415 - val_loss: 0.0025 - val_mae: 0.0395
Epoch 5/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 0.0016 - mae: 0.0286 - val_loss: 7.2408e-04 - val_mae: 0.0213
Epoch 6/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 6.6100e-04 - mae: 0.0209 - val_loss: 3.0660e-04 - val_mae: 0.0143
Epoch 7/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 4.5247e-04 - mae: 0.0180 - val_loss: 4.1521e-04 - val_mae: 0.0170
Epoch 8/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 3.9957e-04 - mae: 0.0164 - val_loss: 8.5259e-04 - val_mae: 0.0231
Epoch 9/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 6.0288e-04 - mae: 0.0170 - val_loss: 4.4972e-04 - val_mae: 0.0162
Epoch 10/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 5.4347e-04 - mae: 0.0166 - val_loss: 7.2052e-04 - val_mae: 0.0242
Epoch 11/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - loss: 3.2453e-04 - mae: 0.0147 - val_loss: 2.4405e-04 - val_mae: 0.0111
Epoch 12/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 2.5882e-04 - mae: 0.0121 - val_loss: 3.3033e-04 - val_mae: 0.0154
Epoch 13/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 1.9749e-04 - mae: 0.0114 - val_loss: 2.0557e-04 - val_mae: 0.0126
Epoch 14/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 1.7784e-04 - mae: 0.0105 - val_loss: 9.5991e-05 - val_mae: 0.0080
Epoch 15/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 1.4564e-04 - mae: 0.0090 - val_loss: 2.9674e-04 - val_mae: 0.0137
Epoch 16/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 1.5958e-04 - mae: 0.0102 - val_loss: 2.7718e-04 - val_mae: 0.0128
Epoch 17/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 1.3619e-04 - mae: 0.0091 - val_loss: 1.5372e-04 - val_mae: 0.0104
Epoch 18/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 1.3715e-04 - mae: 0.0089 - val_loss: 1.4900e-04 - val_mae: 0.0094
Epoch 19/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - loss: 1.0021e-04 - mae: 0.0079 - val_loss: 7.7068e-05 - val_mae: 0.0064
Epoch 20/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 5.4014e-05 - mae: 0.0057 - val_loss: 1.0537e-04 - val_mae: 0.0085
Epoch 21/100
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 8.1230e-05 - mae: 0.0069 - val_loss: 6.2492e-05 - val_mae: 0.0063
Epoch 22/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 7.7189e-05 - mae: 0.0069 - val_loss: 9.4164e-05 - val_mae: 0.0076
Epoch 23/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 8.1990e-05 - mae: 0.0072 - val_loss: 5.0479e-05 - val_mae: 0.0055
Epoch 24/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 5.6215e-05 - mae: 0.0057 - val_loss: 7.9393e-05 - val_mae: 0.0071
Epoch 25/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 5.8064e-05 - mae: 0.0056 - val_loss: 5.9695e-05 - val_mae: 0.0063
Epoch 26/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 3.7396e-05 - mae: 0.0049 - val_loss: 8.7629e-05 - val_mae: 0.0068
Epoch 27/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 6.5430e-05 - mae: 0.0060 - val_loss: 7.0848e-05 - val_mae: 0.0063
Epoch 28/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - loss: 6.8794e-05 - mae: 0.0062 - val_loss: 1.3323e-04 - val_mae: 0.0090
Epoch 29/100
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 1.1329e-04 - mae: 0.0079 - val_loss: 8.4036e-05 - val_mae: 0.0076
```

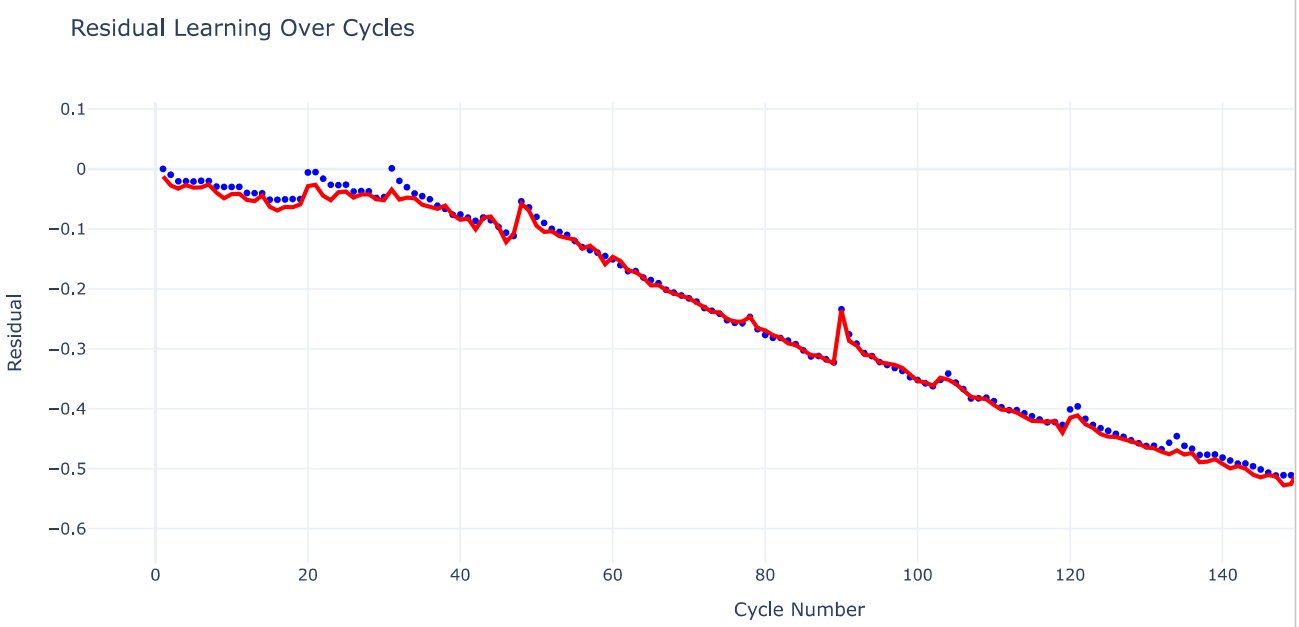
```
fig = go.Figure()

fig.add_trace(go.Scatter(x=np.arange(0, epochs, 1),
                         y=history.history['mae'],
                         mode='lines',
                         name=f'Training MAE',
                         marker_size=3,
                         line_color='orange'))
fig.add_trace(go.Scatter(x=np.arange(0, epochs, 1),
                         y=history.history['val_mae'],
                         mode='lines',
                         name=f'Validation MAE',
                         line_color='grey'))

fig.update_layout(
                  title="Network training",
                  xaxis_title="Epochs",
                  yaxis_title=f"Mean Absolute Error")
fig.update_layout({'plot_bgcolor': '#f2f8fd' ,
                   'paper_bgcolor': 'white',},
                   template='plotly_white')
```




```
pred_all = model.predict(X_in).reshape(-1)

x_vals = np.array(dfb.index)
y_true = np.array(X_out).reshape(-1)
y_pred = np.array(pred_all).reshape(-1)

# zoom the y-axis in around the residual values
y_min = min(y_true.min(), y_pred.min())
y_max = max(y_true.max(), y_pred.max())
```

```
pad = (y_max - y_min) * 0.2

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=x_vals,
    y=y_true,
    mode='markers',
    name='Residuals',
    marker=dict(size=5, color='blue')
))

fig.add_trace(go.Scatter(
    x=x_vals,
    y=y_pred,
    mode='lines',
    name='Predicted Residuals',
    line=dict(color='red', width=3)
))

fig.update_layout(
    title="Residual Learning Over Cycles",
    xaxis_title="Cycle Number",
    yaxis_title="Residual",
    yaxis=dict(range=[y_min - pad, y_max + pad]),
    plot_bgcolor='white',
    paper_bgcolor='white',
    template='plotly_white'
)

fig.show()
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
```




```
print(dfb['C. Capacity'].head(10))
print(dfb['C. Capacity'].min(), dfb['C. Capacity'].max())
```

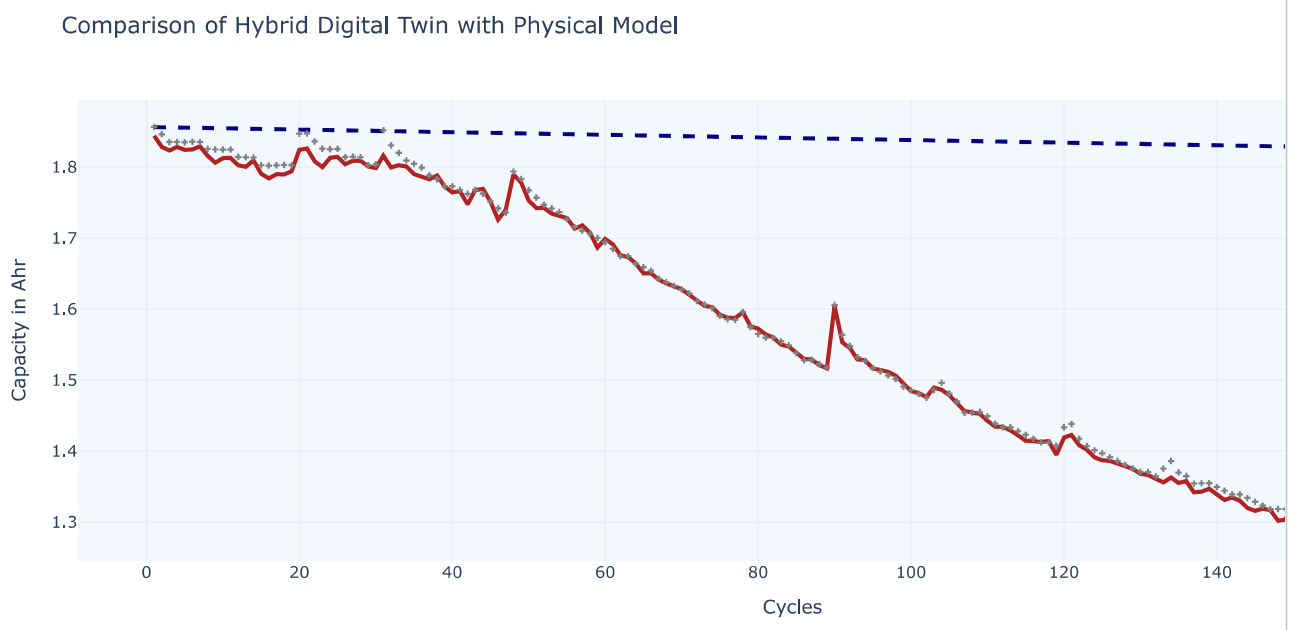
```
id_cycle
1     1.856302
2     1.856116
3     1.855931
4     1.855745
5     1.855559
6     1.855374
7     1.855188
8     1.855003
9     1.854817
10    1.854632
Name: C. Capacity, dtype: float64
1.8255589586645102 1.8563017813582026
```

```
print(len(dfb))
```

```
168
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

pred_all = model.predict(X_in).reshape(-1)
dfb['Digital_Twin_Capacity'] = dfb['C. Capacity'] + pred_all

mae_physical = mean_absolute_error(dfb['Capacity'], dfb['C. Capacity'])
mae_twin = mean_absolute_error(dfb['Capacity'], dfb['Digital_Twin_Capacity'])

rmse_physical = np.sqrt(mean_squared_error(dfb['Capacity'], dfb['C. Capacity']))
rmse_twin = np.sqrt(mean_squared_error(dfb['Capacity'], dfb['Digital_Twin_Capacity']))

print("Physical Model MAE:", round(mae_physical, 3))
print("Digital Twin MAE:", round(mae_twin, 3))
print("Physical Model RMSE:", round(rmse_physical, 3))
print("Digital Twin RMSE:", round(rmse_twin, 3))
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
Physical Model MAE: 0.268
Digital Twin MAE: 0.007
Physical Model RMSE: 0.324
Digital Twin RMSE: 0.01
```

```
pred_all = model.predict(X_in).reshape(-1)

dfb['Digital_Twin_Capacity'] = dfb['C. Capacity'] + pred_all

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=dfb.index,
    y=dfb['Digital_Twin_Capacity'],
    mode='lines',
    name='Hybrid Digital Twin',
    line=dict(color='firebrick', width=3)
))

fig.add_trace(go.Scatter(
    x=dfb.index,
    y=dfb['C. Capacity'],
    mode='lines',
    name='Physical Model',
    line=dict(color='navy', width=3, dash='dash')
))

fig.add_trace(go.Scatter(
    x=dfb.index,
    y=dfb['Capacity'],
    mode='markers',
    marker=dict(size=4, color='grey', symbol='cross'),
    name='Observed Capacity'
))

fig.update_layout(
    title="Comparison of Hybrid Digital Twin with Physical Model",
    xaxis_title="Cycles",
    yaxis_title="Capacity in Ahr",
    plot_bgcolor='#f2f8fd',
    paper_bgcolor='white',
    template='plotly_white'
)

fig.show()
```

```
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
```